WasteWatch — End-to-End Architecture

Container → module → function flow · dashed/green boxes = planned MiniCPM-o narration addition, not yet live

Cameras

RTSP / RTMP / HTTPS(HLS,MJPEG) feeds, 10-50 sources

↓

mediamtx · ingest gateway

Normalizer
Any input protocol → internal RTSP. REST API auto-creates a path per camera on add/edit (called from api's camera routes).

Output
RTSP out → DeepStream (pipeline) for processing
WebRTC out → frontend for live view

→

pipeline · DeepStream GPU process (pipeline.py)

streammux
Batches N camera streams into one GPU batch

nvinfer
RF-DETR (TensorRT FP16), 2-class filtered detector → bbox+class+conf

nvtracker
ByteTrack (Kalman+Hungarian, CPU) → assigns `track_id`

nvdsanalytics
Point-in-polygon ROI filter, drops tracks outside zone

event_fsm.py
`EventFSMManager.update()` — ENTER→OBSERVE→VIOLATION→ALERTING→ESCALATE per track

nvosd
Draws boxes/labels; decoration probe (~pipeline.py:187) colors by FSM state

tee → 3 branches
B1: nvstreamdemux → per-cam RTSP → mediamtx
B2: fakesink_meta → `extract_metadata()` probe → Kafka
B3: fakesink_capture → `capture_frame_probe()` → JPEG cache

vlm_grpc_client.py
`VLMGrpcClient.submit()` — fire-and-forget gRPC call on ENTER/VIOLATION, thread pool + 30s circuit breaker

evidence_writer.py
`start_evidence / add_frame / finalize` — writes evt_*.json + clip on track exit

kafka_producer.py
`PipelineKafkaProducer` — send_metadata/event/vlm_request/lpr/tts/alert

pipeline (planned) · narration sampler

narration_sampler.py
GLib timer @ `WW_NARRATION_FPS` reads existing `frame_jpeg_cache` → `narration_tick()`

narration_client.py
`NarrationClient.submit_frame()` — per-camera session (stub today, real vllm-omni duplex call later)

kafka_producer.py
`send_narration_flag()` → new topic on any flag

→

vlm-worker · gRPC servicer

grpc_server.py
`VLMServicer.Analyze()` — builds VLMRequest, crops via `_crop_to_bbox`, calls client, publishes result

client.py
`VLMClient.analyze()` → `_call_vlm()` HTTP POST /v1/chat/completions; retries once w/ strict prompt on parse fail

worker.py
`VLMWorker` — persistent ffmpeg RTSP frame grabber fallback

↕ HTTP

vlm · model server

Today: **vLLM** + Qwen2.5-VL-7B-AWQ
Planned: **vllm-omni** + MiniCPM-o 4.5 (9B), vision-only (init_audio=False), enable_thinking=False for trigger path

Narration mode (planned): same vllm-omni process, stateful duplex session — Omni-Flow, 1s chunk timeline, TAIL alignment, `streaming_prefill()/streaming_gen` — NOT the same call shape as trigger path

→

KAFKA topics

metadata — per-frame
events — FSM state change
vlm.requests
vlm.results
alerts
lpr
tts
evidence
logs
config.updates
sip
camera.status
narration.flags (planned)

→

alert-svc

dispatcher.py
`AlertDispatcher._handle_message()` — throttle check → fan out (Telegram/WS), skips info/non-dumping

escalation_worker.py
Repeat-offender / ignored-alert escalation logic

evidence-svc

worker.py
`EvidenceWorker` — 2 consumers: evidence files + FSM events (smart recording)

recorder.py
`EvidenceRecorder` — save_snapshot/save_metadata

stream_recorder.py
`StreamRecorder` — pre/post buffer clip start/stop on VIOLATION/EXITED

dropbox_sync.py
`DropboxSync.upload()`

lpr-worker

Consumes `lpr` topic → PaddleOCR on vehicle crop → regex-validate plate → writes `plate_reads`

tts-worker / tts

Consumes `tts` topic → Piper ONNX generates WAV → pushed to speaker + WebSocket

sip-svc

Consumes `sip` /escalation events → phone call via Twilio/SIP gateway

log-worker

Consumes `logs` topic → structured log ingest → Qdrant embeddings

api · FastAPI backend

routes/evidence.py
`list_evidence_files()` scans evt_*.json sidecars → history browser; `get_evidence_file()` serves clip/snapshot (path-guarded)

routes/alerts.py
`list_alerts / acknowledge_alert`

routes/cameras.py, roi.py
Camera CRUD, ROI polygon config → triggers mediamtx path + pipeline reload

WebSocket routers
ws_metadata / ws_events / ws_alerts — live push to frontend

postgres

cameras, violation_events, evidence_files, alerts, track_sessions, vlm_analyses, plate_reads, tts_cache, log_entries + narration_flags (planned)

redis / qdrant / keycloak

redis: cache/session · qdrant: vector log search · keycloak: auth (viewer/admin/owner roles)

↓

Outputs

Telegram bot · Phone call (SIP/Twilio) · IP Speaker (TTS audio) · Frontend dashboard (WebRTC live view + WebSocket live alerts + TimelineView history browser)

health-monitor / healer

Watches `camera.status`; auto-restarts dead streams/services, publishes `healing` / `misses`

reports

Scheduled aggregation over Postgres → PDF/email reports (SMTP)

api (planned addition)

narration_worker.py
`NarrationFlagWorker._process_flag()` — consumes `narration.flags` → resolves camera → inserts `narration_flags` row (review_status=pending)

routes/narration_flags.py
`list / confirm / dismiss` — confirm promotes to `ViolationEvent`